

■ 1. ESTRUCTURA DE PROYECTO

Creamos la carpeta raíz:

```
proyecto/  
├─ back/  
└─ front/
```

- 💡 En esta práctica trabajaremos solo dentro de la carpeta back/.

■ 2. INSTALACIÓN Y CONFIGURACIÓN INICIAL DE NESTJS

2.1 Instalar la última versión de Node.js

Descargar desde: <https://nodejs.org/es/download>

⚠ Importante: comprobar versión

```
node -v  
npm -v
```

Debe ser **Node 20+** y **npm 10+**.

2.2 Instalar el CLI de Nest

```
cd back  
npm install -g @nestjs/cli
```

Crear el proyecto Nest dentro de la carpeta back:

```
nest new .
```

Seleccionar:

```
✓ Which package manager would you ❤️ to use? → npm
```

■ 3. CONFIGURACIÓN DE VARIABLES DE ENTORNO

Instalamos las dependencias:

```
npm install @nestjs/config dotenv
```

Creamos en la raíz de back/ los siguientes archivos:

```
.env  
.stg.env  
.prod.env
```

Contenido base de los *.env*:

```
POSTGRES_DB=postgres  
POSTGRES_USER=postgres  
POSTGRES_PASSWORD=12345678  
POSTGRES_PORT=5432  
POSTGRES_HOST=database-1.c1g2i8qc4akb.us-east-1.rds.amazonaws.com  
  
API_KEY=DEV-KEY-1234  
API_KEY_PROD=PROD1212121SA  
NODE_ENV=dev  
PORT=3000
```

■ 4. MÓDULO DE CONFIGURACIÓN Y BASE DE DATOS

Creamos el módulo de base de datos:

```
cd src
nest generate module database
```

4.1 Configuración de *database.module.ts*

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import config from 'src/config/config';

@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
      load: [config],
      envFilePath:
        process.env.NODE_ENV === 'prod'
          ? '.prod.env'
          : process.env.NODE_ENV === 'stg'
            ? '.stg.env'
            : '.env',
    }),
  ],
})
export class DatabaseModule {}
```

4.2 Crear carpeta config/ con archivo config.ts

Ruta: src/config/config.ts

```
import { registerAs } from '@nestjs/config';

export default registerAs('config', () => ({
  postgres: {
    dbName: process.env.POSTGRES_DB,
    host: process.env.POSTGRES_HOST,
    port: parseInt(process.env.POSTGRES_PORT ?? '5432', 10),
    user: process.env.POSTGRES_USER,
    password: process.env.POSTGRES_PASSWORD,
  },
  api: {
    key: process.env.API_KEY,
    keyProd: process.env.API_KEY_PROD,
  },
  environment: process.env.NODE_ENV || 'dev',
}));
```

■ 5. INTEGRAR TYPEORM CON POSTGRESOL

Instalamos los paquetes:

```
npm install @nestjs/typeorm typeorm pg
```

Modificamos database.module.ts:

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import config from 'src/config/config';
import { ConfigType } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';

@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
      load: [config],
      envFilePath:
        process.env.NODE_ENV === 'prod'
          ? '.prod.env'
          : process.env.NODE_ENV === 'stg'
            ? '.stg.env'
            : '.env',
    }),
    TypeOrmModule.forRootAsync({
      name: 'default',
      inject: [config.KEY],
      useFactory: (cfg: ConfigType<typeof config>) => {
        const { postgres } = cfg;
        return {
          type: 'postgres',
          host: postgres.host,
          port: postgres.port,
          username: postgres.user,
          password: postgres.password,
          database: postgres.dbName,
          synchronize: true,
          autoLoadEntities: true,
          ssl: { rejectUnauthorized: false }, // acepta certificados autofirmados
        };
      },
    }),
  ],
})
export class DatabaseModule {}
```

■ 6. MÓDULO DE PELÍCULAS

Creamos el módulo, controlador y servicio:

```
nest generate module peliculas
nest generate controller peliculas
nest generate service peliculas
```

En peliculas.module.ts añadimos la importación de TypeOrmModule:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { PeliculasController } from './peliculas.controller';
import { PeliculasService } from './peliculas.service';
import { Pelicula } from './entities/pelicula.entity';

@Module({
  imports: [TypeOrmModule.forFeature([Pelicula])],
  controllers: [PeliculasController],
  providers: [PeliculasService],
})
export class PeliculasModule {}
```

■ 7. ENTIDAD PELÍCULA

Creamos: **pelicula.entity.ts**

Ruta: src/peliculas/entities/pelicula.entity.ts

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn, UpdateDateColumn } from 'typeorm';

@Entity({ name: 'PELICULAS1' })
export class Pelicula {
  @PrimaryGeneratedColumn({ name: 'ID' })
  id: number;

  @Column({ type: 'varchar', length: 100, name: 'TITULO' })
  titulo: string;

  @Column({ type: 'int', name: 'ANIO' })
  anio: number;

  @Column({ type: 'text', nullable: true, name: 'DESCRIPCION' })
  descripcion?: string;

  @Column({ type: 'varchar', length: 255, nullable: true, name: 'IMAGEN_URL' })
  imagen_url?: string;

  @CreateDateColumn({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP', name: 'CREATED_AT' })
  createdAt?: Date;

  @UpdateDateColumn({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP', name: 'UPDATED_AT' })
  updateAt?: Date;
}
```

■ 8. DTOS (DATA TRANSFER OBJECTS)

Instalamos validadores:

```
npm i class-validator class-transformer @nestjs/mapped-types
```

Creamos: **create-pelicula.dto.ts**

```
import {
  IsString,
  IsNumber,
  IsOptional,
  IsUrl,
  IsNotEmpty,
  MaxLength,
  Min,
  Max,
} from 'class-validator';
import { Type } from 'class-transformer';

export class CreatePeliculaDto {
  @IsString()
  @IsNotEmpty({ message: 'El título es obligatorio' })
  @MaxLength(100)
  readonly titulo: string;

  @Type(() => Number)
  @IsNumber({}, { message: 'El año debe ser un número' })
  @Min(1888)
  @Max(new Date().getFullYear() + 1)
  readonly anio: number;

  @IsString()
  @IsOptional()
  readonly descripcion?: string;

  @IsUrl()
  @IsOptional()
  readonly imagen_url?: string;
}
```

Creamos: **update-pelicula.dto.ts**

```
import { PartialType } from '@nestjs/mapped-types';
import { CreatePeliculaDto } from './create-pelicula.dto';

export class UpdatePeliculaDto extends PartialType(CreatePeliculaDto) {}
```

■ 9. SERVICIO DE PELÍCULAS

Ruta: src/peliculas/peliculas.service.ts

```
import { Injectable } from '@nestjs/common';
import { Pelicula } from '../entities/pelicula.entity';
import { Repository } from 'typeorm';
import { InjectRepository } from '@nestjs/typeorm';
import { CreatePeliculaDto } from '../dtos/create-pelicula.dto';

@Injectable()
export class PeliculasService {
  constructor(
    @InjectRepository(Pelicula)
    private readonly repo: Repository<Pelicula>,
  ) {}

  findAll() {
    return this.repo.find();
  }

  async create(createPeliculaDto: CreatePeliculaDto) {
    const nuevaPelicula = this.repo.create(createPeliculaDto);
    return this.repo.save(nuevaPelicula);
  }
}
```


■ 10. CONTROLADOR DE PELÍCULAS

Ruta: src/peliculas/peliculas.controller.ts

```
import { Controller, Get, Post, Body } from '@nestjs/common';
import { PeliculasService } from '../peliculas.service';
import { CreatePeliculaDto } from '../dtos/create-pelicula.dto';

@Controller('peliculas')
export class PeliculasController {
  constructor(private peliculasService: PeliculasService) {}

  @Get()
  findAll() {
    return this.peliculasService.findAll();
  }

  @Post()
  create(@Body() createPeliculaDto: CreatePeliculaDto) {
    return this.peliculasService.create(createPeliculaDto);
  }
}
```

■ 11. EJECUCIÓN POR ENTORNO

```
NODE_ENV=dev npm run start
NODE_ENV=stg npm run start
NODE_ENV=prod npm run start
```

■ 12. TEST DE ENDPOINTS CRUD

Crear película

POST `http://localhost:3000/peliculas`

Body (JSON):

```
{
  "titulo": "Interstellar",
  "anio": 2014,
  "descripcion": "Exploración espacial a través de agujeros de gusano",
  "imagen_url": "https://example.com/interstellar.jpg"
}
```

Listar películas

GET `http://localhost:3000/peliculas`

Ejemplo de respuesta:

```
[
  {
    "id": 1,
    "titulo": "Interstellar",
    "anio": 2014,
    "descripcion": "Exploración espacial a través de agujeros de gusano",
    "imagen_url": "https://example.com/interstellar.jpg"
  }
]
```

■ 13. SOLUCIÓN DE ERRORES COMUNES

Si aparecen errores con TypeORM o dependencias, ejecutar:

```
npm install typeorm@0.3.16 --save
rm -r node_modules
npm ci
npm run start
```

